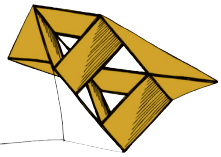
FACULTY OF APPLIED SCIENCES
UNIVERSITY
OF WEST BOHEMIA

STUDENT
SCIENTIFIC CONFERENCE 2026



Distributed Metasolver

Bc. Jonáš Dufek[1]

1 Introduction

Many modern applications, such as artificial intelligence, machine learning, or medical informatics, are centered around **optimizing** the parameters of specialized computational models. This computationally demanding process is typically handled by solver software, the intent of which is to execute an optimization algorithm and return the optimal parameters.

A novel approach to this concept is using multiple optimization algorithms instead of a single one, solvers designed in this way are called **metasolvers**. Furthermore, the solver presented here is distributed, making it possible to distribute the workload across many heterogeneous computational nodes via the **island-based model**. These two ideas thus complement each other, metasolving has the potential of improving the quality and diversity of solutions, whereas the distributed island model improves the overall execution speed.

This distributed metasolver was implemented as a C++ library, which allows it to be easily integrated into existing applications. Example of such an application is the SmartCGMS framework [2], where a unified solver interface can be used to connect any third party solver, allowing it to be used for the optimization of the framework's various glucose prediction models.

The performance characteristics of this library were evaluated on several standardized optimization problems. The results have demonstrated exceptional performance regarding reliability and distribution of work, although the load-balancing mechanism proved to be less efficient in certain metasolving scenarios.

2 Population-Based Optimization

Traditional optimization algorithms, such as gradient descent, maintain a single candidate solution which is iteratively improved until an optimum of the optimized function is reached. This work utilizes population-based optimization algorithms, which differ from classical methods in the fact that they utilize a "**population**" of candidate solutions of an optimization problem, rather than trying to iteratively improve a single one.

These algorithms are especially beneficial for parallel and distributed implementations, as different portions of the population can be optimized independently on multiple processing elements or nodes.

3 Island Model

The island model is a collection of islands connected by a topology, each island maintains a population of individuals and executes an optimization algorithm.

The topology can be visualized as a graph, where each edge represents a communication

[1] Master's programme in Applied Sciences, Distributed Computing Systems, Dept. of Computer Science and Engineering, Faculty of Applied Sciences, UWB Pilsen, jonasd@students.zcu.cz

channel over which islands exchange individuals using the so-called process of **migration**.

As highlighted by [3], migration is an important part of this process, because it increases the diversity of individuals in each island and thus allows the algorithms to cooperate.

4 System Architecture

The key dependencies of this software are Pagmo [1] and ZeroMQ [4]. Pagmo provides us with optimization algorithms and the "Archipelago-Island-UDI" model. ZeroMQ is a networking library using a modern asynchronous event oriented pattern, together with a transport protocol leveraging useful features such as *multi-part messages*.

The most important classes of the implemented library are `distributed_controller`, `distributed_worker` and `distributed_island`. Refer to figure 1 for further details.

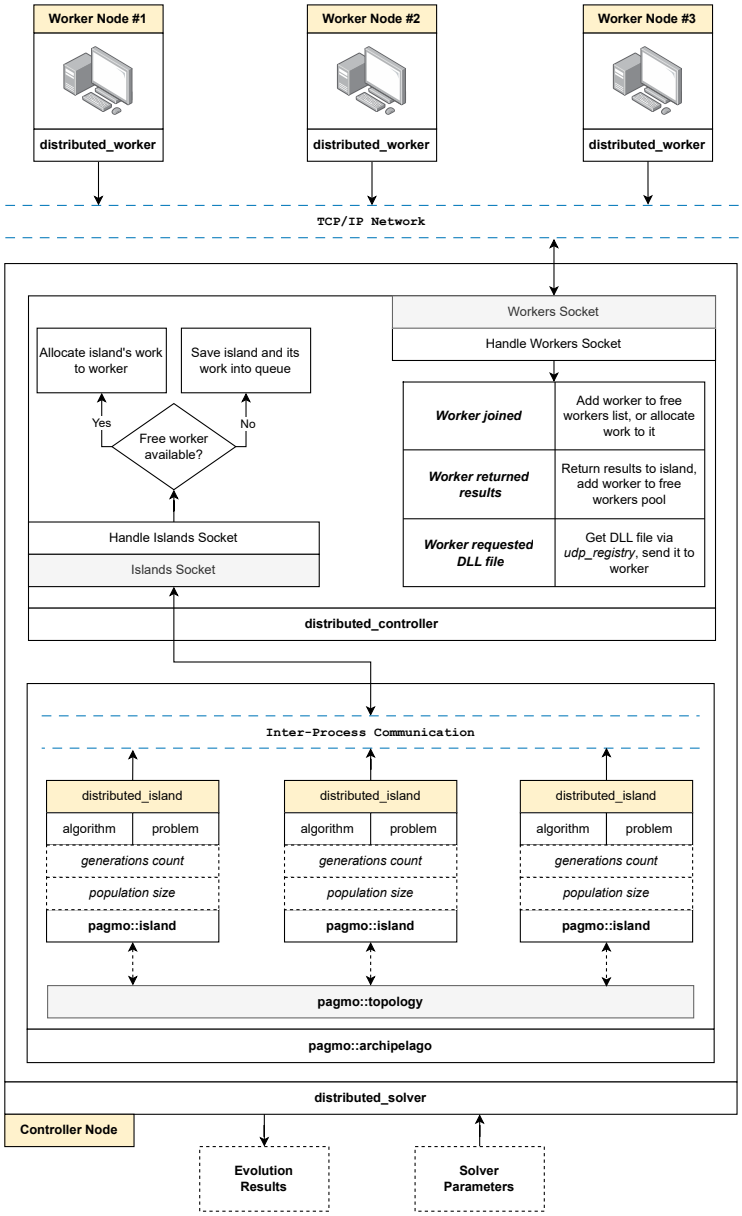


Figure 1: Architecture of the metasolver distributed system, showcasing three connected workers.

5 Experimental Results

The library was evaluated in six different scenarios covering distributed single-algorithm solving, distributed metasolving, and local metasolving on standard benchmark problems provided by the Pagmo library (Rastrigin, Rosenbrock, Ackley, Griewank, Schwefel, ZDT1 to ZDT4).

The distributed single-algorithm scenario yielded more than adequate results, achieving up to **115%** faster completion times and **24%** better fitness compared to a local solver, with the load-balancing mechanism successfully absorbing a third low-performance node without degrading the cluster's total computational power.

The metasolving scenarios produced mixed results, when all of the used algorithms had similar computational requirements, the metasolver's performance was comparable to the best individual algorithms. However, in some scenarios such as CMA-ES algorithm on the Rosenbrock function, the load-balancing mechanism was unable to prevent the slower algorithm from becoming a bottleneck.

6 Conclusion

The primary objectives of this work were to create and implement a distributed metasolver, together with assessing its performance.

The distributed controller-worker architecture proved to be efficient and reliable when used as part of the Island-Archipelago model. On the other hand, the metasolving performance turned out to be sensitive to the selection of parameters, algorithms and optimization problems.

Future work could include for example improving the Pagmo library‘s metasolving capabilities[1], or implementing more robust load-balancing mechanisms.

References

[1] Francesco Biscani and Dario Izzo. A parallel global multiobjective framework for optimization: pagmo. *Journal of Open Source Software*, 5(53):2338, 2020.

[2] Tomas Koutny and Martin Ubl. Smartcgms as a testbed for a blood-glucose level prediction and/or control challenge with (an fda-accepted) diabetic patient simulation. *Procedia Computer Science*, 177:354–362, 2020. The 11th International Conference on Emerging Ubiquitous Systems and Pervasive Networks (EUSPN 2020) / The 10th International Conference on Current and Future Trends of Information and Communication Technologies in Healthcare (ICTH 2020) / Affiliated Workshops.

[3] Fei Peng, Ke Tang, Guoliang Chen, and Xin Yao. Population-based algorithm portfolios for numerical optimization. *IEEE Transactions on Evolutionary Computation*, 14(5):782–805, October 2010.

[4] ZeroMQ contributors. Zeromq: High-performance asynchronous messaging library. `https://zeromq.org/`, 2026. Accessed: 2026-05-03.

[1] Helpful might be limiting the algorithm's execution time by the amount of consumed CPU time, not by the number of generations it's allowed to evolve. This would solve the issue of different algorithms having different per-generation computational requirements.